

Backend Plan & Variable Contract

A full Django backend specification with a frozen shared contract — so two developers can build different phases in parallel and merge with zero conflicts.

WALAA · LOYALTY & WALLET-PASS PLATFORM

Phase 1 — You · Phase 2 — Joe

Stack

Django REST · Celery

Data

Postgres · Redis

Contract

Frozen in Phase 0

Project: Walaa — digital loyalty & wallet-pass platform (Django REST backend) **Team:** 2 developers — **You** (Phase 1, plus infra & wallets) · **Joe** (Phase 2, product engine) **Purpose of this document:** a single source of truth so that two people can build different phases *in parallel*, using *identical* names for every model field, enum value, endpoint key, function signature, env var, and task — so that when the branches merge, **there are no conflicts and nothing is renamed twice**.

The golden rule: Everything in **Section 3 (The Variable Contract)** is *frozen in Phase 0*. After that, both developers **import** these names from the shared modules — they never re-declare them locally. Any change to a contract name is a **joint PR to this file first**, then code. This is the mechanism that makes parallel work conflict-free.

Table of contents

- 1. Stack & architecture
- 2. Repository layout & ownership
- 3. **The Variable Contract** (frozen) - 3.1 Naming conventions - 3.2 Shared enums — `core/enums.py` - 3.3 Shared constants — `core/constants.py` - 3.4 Data model — every model, every field - 3.5 Internal service interfaces — exact signatures - 3.6 REST API contract — endpoints & JSON keys - 3.7 Response envelope, pagination & error codes - 3.8 Environment variables - 3.9 Celery queues & task names - 3.10 Settings keys
- 4. Phases (0–5) with owners, deliverables & exit criteria
- 5. How Phase 1 & Phase 2 merge with zero conflict (worked example)
- 6. Testing & Definition of Done
- 7. Appendix — canonical-name cheat sheet

1. Stack & architecture

Layer	Choice
Framework	Django + Django REST Framework
DB	PostgreSQL
Cache / broker	Redis
Async	Celery + Celery beat
API schema	drf-spectacular (OpenAPI from code)
Auth	JWT (<code>django-rest-framework-simplejwt</code>)
Wallets	<code>google-auth</code> + <code>PyJWT</code> (Google) · <code>cryptography</code> / <code>openssl</code> + <code>httpx</code> HTTP/2 (Apple)
Quality	<code>ruff</code> · <code>black</code> · <code>mypy</code> · <code>pytest</code> · <code>factory_boy</code>

Runtime topology: one Django web process (Gunicorn/Uvicorn), one Celery worker, one Celery beat, Postgres, Redis. The Apple Wallet web service lives inside the same Django app but must be reachable on the public HTTPS host.

The core principle: all balance mutations flow through **one** module — `core/ledger.py`. No endpoint writes to `StampLedger` or `CustomerCard.stamp_count` directly. This single chokepoint is what keeps the two phases consistent.

2. Repository layout & ownership

```
backend/
  config/          settings, urls, celery app          [Phase 0 · shared]
  core/            models · enums · constants · auth · ledger · tenancy  [Phase 0 · shared,
FROZEN]
  common/          base serializers · pagination · errors · permissions  [Phase 0 · shared]
  enrollment/      join flow · enrollment tokens · consent  [Phase 1 · You]
  wallets/         interfaces · google/ · apple/ · webservice/ · apns · tasks  [Phase 1 · You]
  loyalty/         stamp · redeem · anti-fraud            [Phase 2 · Joe]
  dashboard/       analytics · merchant/staff/location config  [Phase 3 · Joe]
  billing/         paymob · fawry · webhooks              [Phase 4 · You]
  messaging/       whatsapp · celery senders              [Phase 4 · You]
  contracts/
    openapi.yaml   generated + frozen v1                  [Phase 0 · shared]
```

Ownership = conflict avoidance. Each Django app has exactly one owner. You only edit your own apps. The three shared areas (`core/`, `common/`, `contracts/openapi.yaml`) change **only** through a PR the other person reviews. Each app owns its own `migrations/` folder; the shared `core/` migrations are created in Phase 0 and then frozen.

3. The Variable Contract (FROZEN)

Everything below is created in **Phase 0** and imported everywhere else. **Do not redefine these names in your own app.** Import them.

3.1 Naming conventions

Thing	Convention	Example
Model fields	snake_case	customer_phone, stamp_count
Primary keys	id, UUID v4	id = UUIDField(primary_key=True)
Foreign keys (attr)	<model>; DB col <model>_id	customer_card, customer_card_id
Enum classes	PascalCase, TextChoices	LedgerEvent
Enum values (DB)	UPPER_SNAKE strings	"STAMP", "REDEEM"
Constants	UPPER_SNAKE	STAMP_COOLDOWN_SECONDS
Functions	snake_case, verb-first	add_stamp(), current_balance()
JSON keys (our API)	snake_case	"customer_card_id"
JSON keys (Apple WS)	Apple's camelCase (external)	"pushToken", "serialNumbers"
Env vars	UPPER_SNAKE	GOOGLE_WALLET_ISSUER_ID
Celery tasks	<app>.tasks.<verb_noun>	wallets.tasks.push_pass_update
Timestamps	created_at, updated_at, <verb>_at	enrolled_at, consent_at

3.2 Shared enums — core/enums.py

```
# core/enums.py — IMPORT FROM HERE. NEVER REDEFINE THESE VALUES.
```

```
from django.db import models
```

```
class Role(models.TextChoices):
    OWNER = "OWNER", "Owner"
    ADMIN = "ADMIN", "Admin"
    SCANNER = "SCANNER", "Scanner"
```

```
class MerchantStatus(models.TextChoices):
    ACTIVE = "ACTIVE", "Active"
    SUSPENDED = "SUSPENDED", "Suspended"
```

```
class PlanTier(models.TextChoices):
    FREE = "FREE", "Free"
    STARTER = "STARTER", "Starter"
    GROWTH = "GROWTH", "Growth"
    CHAIN = "CHAIN", "Chain / Custom"
```

```
class CardType(models.TextChoices):
    STAMP = "STAMP", "Stamp card"
    POINTS = "POINTS", "Points card"
```

```

class CardStatus(models.TextChoices):
    DRAFT = "DRAFT", "Draft"
    ACTIVE = "ACTIVE", "Active"
    ARCHIVED = "ARCHIVED", "Archived"

class CustomerCardStatus(models.TextChoices):
    ACTIVE = "ACTIVE", "Active"
    COMPLETED = "COMPLETED", "Completed"
    BLOCKED = "BLOCKED", "Blocked"

class LedgerEvent(models.TextChoices):
    ENROLL = "ENROLL", "Enrollment"
    STAMP = "STAMP", "Stamp added"
    REDEEM = "REDEEM", "Reward redeemed"
    ADJUST = "ADJUST", "Manual adjustment"

class WalletPlatform(models.TextChoices):
    APPLE = "APPLE", "Apple Wallet"
    GOOGLE = "GOOGLE", "Google Wallet"

class RedemptionStatus(models.TextChoices):
    PENDING = "PENDING", "Pending"
    CLAIMED = "CLAIMED", "Claimed"
    VOID = "VOID", "Void"

```

3.3 Shared constants — `core/constants.py`

```

# core/constants.py — single home for tunables both phases rely on.

# Anti-fraud (used by loyalty/, enforced server-side)
STAMP_COOLDOWN_SECONDS = 30 # min seconds between stamps on one card
MAX_STAMPS_PER_CARD_PER_DAY = 12
MAX_STAMPS_PER_STAFF_PER_MIN = 20

# Tokens
AUTH_TOKEN_BYTES = 24 # CustomerCard.auth_token entropy
ENROLL_TOKEN_BYTES = 16

# Enrollment
ENROLL_TOKEN_TTL_DAYS = None # None = never expires

# Wallet
PASS_BARCODE_PREFIX = "WLA" # barcode payload prefix

# Pagination
DEFAULT_PAGE_SIZE = 25
MAX_PAGE_SIZE = 100

```

3.4 Data model — every model, every field

All models live in `core/models.py` (Phase 0, frozen). FK attribute names are fixed; the DB column is `<attr>_id`. All have `id: UUID (pk)`.

Merchant

Field	Type	Notes
<code>id</code>	UUID pk	
<code>name</code>	char(120)	display name
<code>slug</code>	slug, unique	url-safe
<code>legal_name</code>	char(160), blank	
<code>status</code>	enum <code>MerchantStatus</code>	default <code>ACTIVE</code>
<code>plan</code>	enum <code>PlanTier</code>	default <code>FREE</code>
<code>logo_url</code>	url, blank	
<code>color_bg</code>	char(7)	hex <code>#RRGGBB</code>
<code>color_fg</code>	char(7)	hex
<code>created_at</code> / <code>updated_at</code>	datetime	auto

Location

Field	Type	Notes
<code>id</code>	UUID pk	
<code>merchant</code>	FK → Merchant	
<code>name</code>	char(120)	
<code>address</code>	char(255), blank	
<code>lat</code> / <code>lng</code>	float, null	
<code>created_at</code> / <code>updated_at</code>	datetime	

StaffUser

Field	Type	Notes
<code>id</code>	UUID pk	
<code>merchant</code>	FK → Merchant	tenant scope
<code>user</code>	OneToOne → <code>auth.User</code>	login identity
<code>role</code>	enum <code>Role</code>	
<code>location</code>	FK → Location, null	scanner's branch
<code>is_active</code>	bool	default <code>True</code>
<code>created_at</code>	datetime	

Card *(program template)*

Field	Type	Notes
<code>id</code>	UUID pk	
<code>merchant</code>	FK → Merchant	
<code>type</code>	enum <code>CardType</code>	default <code>STAMP</code>
<code>name</code>	char(120)	
<code>stamps_required</code>	int	stamps to complete (stamp cards)
<code>reward_title</code>	char(120)	
<code>reward_description</code>	text, blank	
<code>color_bg</code> / <code>color_fg</code>	char(7)	overrides merchant brand
<code>logo_url</code>	url, blank	
<code>google_class_id</code>	char(120), blank	set after provisioning
<code>status</code>	enum <code>CardStatus</code>	default <code>DRAFT</code>
<code>created_at</code> / <code>updated_at</code>	datetime	

CustomerCard *(per-customer instance — `id` is the wallet serial)*

Field	Type	Notes
<code>id</code>	UUID pk	= wallet serialNumber
<code>card</code>	FK → Card	
<code>merchant</code>	FK → Merchant	denormalized for scoping
<code>customer_phone</code>	char(20)	E.164
<code>customer_name</code>	char(120), blank	
<code>stamp_count</code>	int	cached balance (derived from ledger)
<code>auth_token</code>	char(48)	per-pass secret (Apple WS auth)
<code>status</code>	enum <code>CustomerCardStatus</code>	default <code>ACTIVE</code>
<code>consent_at</code>	datetime, null	PDPL consent timestamp
<code>enrolled_at</code>	datetime	
<code>last_event_at</code>	datetime, null	last stamp/redeem
<code>created_at</code> / <code>updated_at</code>	datetime	

Uniqueness: (`card`, `customer_phone`) unique together — one card per phone per program.

StampLedger *(append-only — never updated or deleted)*

Field	Type	Notes
id	UUID pk	
customer_card	FK → CustomerCard	
merchant	FK → Merchant	
event_type	enum LedgerEvent	
delta	int	+1 stamp, -N on redeem, etc.
balance_after	int	snapshot after applying delta
staff	FK → StaffUser, null	who issued
location	FK → Location, null	where
note	char(255), blank	
created_at	datetime	

Reward

Field	Type	Notes
id	UUID pk	
card	FK → Card	
title	char(120)	
description	text, blank	
threshold	int	stamps required to unlock
is_active	bool	default True
created_at	datetime	

Redemption

Field	Type	Notes
id	UUID pk	
customer_card	FK → CustomerCard	
reward	FK → Reward	
merchant	FK → Merchant	
staff	FK → StaffUser, null	
location	FK → Location, null	
status	enum RedemptionStatus	default CLAIMED
created_at	datetime	

EnrollmentToken

Field	Type	Notes
id	UUID pk	
merchant	FK → Merchant	
card	FK → Card	which program the QR enrolls into
token	char(32), unique	QR payload
expires_at	datetime, null	
is_active	bool	default <code>True</code>
created_at	datetime	

WalletRegistration

Field	Type	Notes
id	UUID pk	
customer_card	FK → CustomerCard	
platform	enum <code>WalletPlatform</code>	
device_library_id	char(64), blank	Apple device id
push_token	char(255), blank	Apple APNs token
google_object_id	char(120), blank	Google object ref
is_active	bool	default <code>True</code>
created_at / updated_at	datetime	

3.5 Internal service interfaces — exact signatures

These are the **seams between the two phases**. Joe codes against them while you implement them. Defined in Phase 0; bodies filled later.

```
# core/ledger.py — THE ONLY way to change a balance. Both phases import these.

def record_enrollment(customer_card) -> "StampLedger": ...

def add_stamp(customer_card, *, staff=None, location=None,
              delta: int = 1, note: str = "") -> "StampLedger":
    """Append a STAMP event, update cached stamp_count, return the ledger row.
    Raises CooldownActive / RateLimited (see common/errors.py)."""

def redeem_reward(customer_card, reward, *, staff=None,
                  location=None) -> "Redemption":
    """Append a REDEEM event + create Redemption. Raises RewardNotReady."""

def current_balance(customer_card) -> int: ...
```

```
def is_reward_ready(customer_card) -> bool: ...
```

```
# wallets/interfaces.py – provisioning + live updates. You implement; Joe calls.
```

```
from typing import Protocol, Optional
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class ProvisionResult:
```

```
    apple_pass_url: Optional[str]
```

```
    google_save_url: Optional[str]
```

```
class WalletProvisioner(Protocol):
```

```
    def provision(self, customer_card) -> ProvisionResult: ...
```

```
class WalletUpdater(Protocol):
```

```
    def push_update(self, customer_card) -> None: ...
```

```
# wallets/service.py – the façade both phases import (dispatches to Apple+Google)
```

```
def provision(customer_card) -> "ProvisionResult": ...
```

```
def push_update(customer_card) -> None: ... # enqueues wallets.tasks.push_pass_update
```

Contract rule: `loyalty/` calls `core.ledger.add_stamp(...)` then `wallets.service.push_update(card)`. It must never import from `wallets/apple/` or `wallets/google/` directly — only the façade.

3.6 REST API contract — endpoints & JSON keys

Base path: `/api/v1/`. All request/response bodies use the exact `snake_case` keys below. Auth via

Authorization: Bearer <access> unless marked *public*.

Auth

```
POST /api/v1/auth/token
```

```
req: { "email": str, "password": str }
```

```
res: { "access": str, "refresh": str }
```

```
POST /api/v1/auth/refresh
```

```
req: { "refresh": str }
```

```
res: { "access": str }
```

Enrollment (*Phase 1 • You*)

```
GET /api/v1/enroll/{token} (public)
```

```
res: { "merchant_name": str, "card_name": str, "reward_title": str,  
      "stamps_required": int, "color_bg": str, "color_fg": str,  
      "logo_url": str }
```

```
POST /api/v1/enroll/{token} (public)
```

```
req: { "customer_phone": str, "customer_name": str, "consent": bool }
```

```
res: { "customer_card_id": uuid, "stamp_count": int,
```

```
"stamps_required": int, "apple_pass_url": str|null,
"google_save_url": str|null }
```

Loyalty *(Phase 2 · Joe)*

```
POST /api/v1/loyalty/stamp
  req: { "customer_card_id": uuid, "delta": int = 1 }
  res: { "customer_card_id": uuid, "stamp_count": int,
        "stamps_required": int, "reward_ready": bool }

POST /api/v1/loyalty/redeem
  req: { "customer_card_id": uuid, "reward_id": uuid }
  res: { "redemption_id": uuid, "status": str, "stamp_count": int }

GET /api/v1/loyalty/cards/{customer_card_id}
  res: { "customer_card_id": uuid, "customer_name": str,
        "stamp_count": int, "stamps_required": int,
        "reward_ready": bool, "status": str }
```

Dashboard *(Phase 3)*

```
GET/POST/PATCH /api/v1/cards          (Card CRUD)
GET             /api/v1/customers      (CustomerCard list, filterable)
GET/POST       /api/v1/staff
GET/POST       /api/v1/locations
GET /api/v1/analytics/summary
  res: { "enrollments": int, "active_cards": int, "redemptions": int,
        "apple_count": int, "google_count": int, "repeat_rate": float }
```

Apple Wallet web service *(Phase 1 · You — Apple's external spec, camelCase)*

```
POST /api/v1/wallet/apple/v1/devices/{device_library_id}/registrations/{pass_type_id}/{serial}
GET   /api/v1/wallet/apple/v1/devices/{device_library_id}/registrations/{pass_type_id}?
      passesUpdatedSince=
GET   /api/v1/wallet/apple/v1/passes/{pass_type_id}/{serial}
DELETE /api/v1/wallet/apple/v1/devices/{device_library_id}/registrations/{pass_type_id}/{serial}
POST  /api/v1/wallet/apple/v1/log
```

These use Apple's field names (`pushToken` , `serialNumbers` , `lastUpdated`) — they are an **external** contract; do not snake_case them.

3.7 Response envelope, pagination & error codes

Success: the resource JSON directly (DRF style). **List endpoints** use cursor pagination:

```
{ "next": "url|null", "previous": "url|null", "results": [ ... ] }
```

Errors always use this shape (`common/errors.py`):

```
{ "error": { "code": "VALIDATION_ERROR", "message": "human text",
            "fields": { "customer_phone": ["This field is required."] } } }
```

Error code enum (`common/errors.py` , frozen):

```
class ErrorCode:
    VALIDATION_ERROR      = "VALIDATION_ERROR"      # 400
    UNAUTHENTICATED       = "UNAUTHENTICATED"       # 401
    PERMISSION_DENIED      = "PERMISSION_DENIED"     # 403
    NOT_FOUND              = "NOT_FOUND"             # 404
    CONFLICT               = "CONFLICT"              # 409
    ALREADY_ENROLLED       = "ALREADY_ENROLLED"      # 409
    TOKEN_EXPIRED          = "TOKEN_EXPIRED"         # 410
    COOLDOWN_ACTIVE        = "COOLDOWN_ACTIVE"       # 429
    RATE_LIMITED           = "RATE_LIMITED"          # 429
    REWARD_NOT_READY      = "REWARD_NOT_READY"     # 422
    WALLET_PROVISION_FAILED = "WALLET_PROVISION_FAILED" # 502
    SERVER_ERROR           = "SERVER_ERROR"          # 500
```

Custom exceptions raised by `core/ledger.py` map to these: `CoolDownActive→COOLDOWN_ACTIVE` ,
`RateLimited→RATE_LIMITED` , `RewardNotReady→REWARD_NOT_READY` .

3.8 Environment variables

```
# --- Django ---
DJANGO_SETTINGS_MODULE=config.settings.prod
SECRET_KEY=
DEBUG=false
ALLOWED_HOSTS=api.walaa.app
BASE_URL=https://api.walaa.app      # used to build webServiceURL & save URLs

# --- Data ---
DATABASE_URL=postgres://...
REDIS_URL=redis://...
CELERY_BROKER_URL=redis://...
CELERY_RESULT_BACKEND=redis://...

# --- Apple Wallet ---
APPLE_TEAM_ID=
APPLE_PASS_TYPE_ID=pass.app.walaa.loyalty
APPLE_PASS_CERT_PATH=/secrets/pass.p12
APPLE_PASS_CERT_PASSWORD=
APPLE_WWDR_CERT_PATH=/secrets/wwdr.pem
APNS_USE_SANDBOX=false

# --- Google Wallet ---
GOOGLE_WALLET_ISSUER_ID=
GOOGLE_SA_KEY_PATH=/secrets/google-sa.json

# --- Tokens ---
WALLET_AUTH_TOKEN_SECRET=
```

```
PASS_BARCODE_SECRET=

# --- Messaging / Billing (Phase 4) ---
WHATSAPP_API_TOKEN=
WHATSAPP_PHONE_ID=
PAYMOB_API_KEY=
FAWRY_MERCHANT_CODE=
FAWRY_SECURITY_KEY=

# --- Ops ---
SENTRY_DSN=
```

3.9 Celery queues & task names

Queues: `default`, `wallet`, `messaging`.

```
# Canonical task names — call by name, never duplicate.
wallets.tasks.provision_pass(customer_card_id: str)           # queue: wallet
wallets.tasks.push_pass_update(customer_card_id: str)        # queue: wallet
wallets.tasks.sync_google_class(card_id: str)                 # queue: wallet
core.tasks.recompute_balance(customer_card_id: str)           # queue: default
messaging.tasks.send_whatsapp(customer_card_id: str,
                                template: str, context: dict) # queue: messaging
```

3.10 Settings keys (custom, in `config/settings`)

```
WALLET = {
    "APPLE": { "PASS_TYPE_ID": env("APPLE_PASS_TYPE_ID"),
               "TEAM_ID": env("APPLE_TEAM_ID") },
    "GOOGLE": { "ISSUER_ID": env("GOOGLE_WALLET_ISSUER_ID") },
}
STAMP_COOLDOWN_SECONDS = constants.STAMP_COOLDOWN_SECONDS
REST_FRAMEWORK = { ...cursor pagination, JWT auth, custom exception handler... }
```

4. Phases

Each phase lists: owner · modules owned · depends on · contract symbols consumed · deliverables · exit criteria · migration notes.

Phase 0 — Foundation & Contract (*shared — build FIRST, then freeze*)

- **Owner:** built together (or by You), then frozen. Nothing else starts until this is tagged `core-v1`.
- **Modules:** `config/`, `core/`, `common/`, `contracts/`.
- **Deliverables:**
 - Project scaffold, split settings, Postgres + Redis + Celery wired.
 - `core/enums.py`, `core/constants.py`, **all** models from §3.4, initial `core` migration.
 - JWT auth, RBAC permissions (`common/permissions.py`), tenant-scoping manager/mixin.

- `core/ledger.py` implemented (the shared mutation API) + its custom exceptions.
- `wallets/interfaces.py` + `wallets/service.py` stubs (signatures only).
- `common/` base serializer, cursor pagination, error envelope + `ErrorCode`.
- drf-spectacular wired; **generate and freeze** `contracts/openapi.yaml` v1.
- **Exit criteria:** migrations apply; you can create Merchant→Card→CustomerCard and call `ledger.add_stamp / redeem_reward` in a shell with correct balances + tenant isolation; OpenAPI committed; tag `core-v1`.
- **Migration notes:** this is the only time `core/` migrations are authored freely. After the tag, `core` schema changes require a joint PR.

Phase 1 — Enrollment + Wallet Provisioning (You)

- **Owner:** You. **Modules:** `enrollment/`, `wallets/`.
- **Depends on:** `core-v1`.
- **Contract symbols consumed:** `core.models.*`, `core.enums.*`, `core.ledger.record_enrollment`, `common.*`, `wallets.interfaces.*`. **Implements** `wallets.service.provision/push_update` + both platform backends.
- **Deliverables:**
 - `EnrollmentToken` issuance + the `GET/POST /enroll/{token}` endpoints (§3.6); PDPL consent → `CustomerCard.consent_at`.
 - Enrollment calls `core.ledger.record_enrollment` and `wallets.service.provision`.
 - **Google:** `LoyaltyClass` per Card, `LoyaltyObject` per CustomerCard, RS256 save URL; `wallets.tasks.provision_pass`, `sync_google_class`.
 - **Apple:** `.pkpass` build + PKCS#7 signing; the 5 web-service endpoints; APNs empty-push via `wallets.tasks.push_pass_update`; `WalletRegistration` lifecycle.
- **Exit criteria:** scanning a token enrolls a customer and returns working `apple_pass_url` + `google_save_url`; a stamp recorded via `ledger` triggers a live pass update on both platforms.
- **Migration notes:** migrations only in `enrollment/` and `wallets/` (e.g. `WalletRegistration` indexes). No `core` edits.

Phase 2 — Loyalty Engine + Anti-Fraud (Joe)

- **Owner:** Joe. **Modules:** `loyalty/`.
- **Depends on:** `core-v1` (can start the moment it's tagged — **parallel with Phase 1**).
- **Contract symbols consumed:** `core.ledger.add_stamp/redeem_reward/current_balance/is_reward_ready`, `core.enums.*`, `core.constants.STAMP_COOLDOWN_SECONDS` etc., `wallets.service.push_update` (**interface — uses a fake until Phase 1 lands**), `common.*`.
- **Deliverables:**
 - `POST /loyalty/stamp`, `POST /loyalty/redeem`, `GET /loyalty/cards/{id}` (§3.6).
 - Anti-fraud enforced **inside** `core/ledger.py` callers + **loyalty validators:** cooldown (`STAMP_COOLDOWN_SECONDS`), per-staff/-card/day limits, staff+location binding written to `StampLedger`.
 - Reward-ready logic via `core.ledger.is_reward_ready`.
 - Every successful stamp/redeem calls `wallets.service.push_update(card)`.
- **Exit criteria:** stamp/redeem update the ledger + cached `stamp_count` correctly; fraud guards reject abuse with the right `ErrorCode`; against a fake `push_update`, the loop is correct; against the real one (post-integration) the pass updates live.
- **Migration notes:** `loyalty/` migrations only (e.g. fraud audit indexes). No `core` edits.

Phase 3 — Dashboard & Analytics API

- **Owner:** Joe (or either). **Modules:** `dashboard/`.
- **Deliverables:** Card CRUD (re-provisions Google class on change via `wallets.tasks.sync_google_class`), staff/location/customer endpoints, `GET /analytics/summary` (aggregations over `StampLedger` + `Redemption` + `WalletRegistration` for the Apple/Google split).
- **Exit criteria:** a merchant can be fully configured and measured via the API.

Phase 4 — Billing + Messaging (You)

- **Owner:** You. **Modules:** `billing/`, `messaging/`.
- **Deliverables:** Paymob/Fawry subscriptions + webhooks; plan gating on `Merchant.plan`; WhatsApp Business API senders (`messaging.tasks.send_whatsapp`) for reward-ready/expiry; metering.
- **Exit criteria:** EGP billing live; WhatsApp notifications fire and are metered per plan.

Phase 5 — Hardening, Observability & Scale (You — *infra*)

- **Deliverables:** Sentry + structured logs + metrics; backups (off-box `pg_dump`); secret rotation; edge rate-limits; DB indexing & Celery scaling; K8s migration when load justifies.
- **Exit criteria:** production-grade with monitoring, alerting, backups verified.

5. How Phase 1 & Phase 2 merge with zero conflict

A concrete walk-through of the parallel window (you on Phase 1, Joe on Phase 2, simultaneously).

Why files never collide

	You (Phase 1)	Joe (Phase 2)
Edits files in	<code>enrollment/</code> , <code>wallets/</code>	<code>loyalty/</code>
Adds URLs under	<code>/enroll/...</code> , <code>/wallet/apple/...</code>	<code>/loyalty/...</code>
Migrations in	<code>enrollment/migrations/</code> , <code>wallets/migrations/</code>	<code>loyalty/migrations/</code>
Imports from <code>core</code> & <code>common</code>	read-only	read-only
Implements	<code>wallets.service.*</code>	nothing in <code>wallets/</code>

Because the two of you only **write** inside disjoint app folders and only **read** from the frozen `core/` + `common/`, Git sees changes in different files → **no merge conflict**. The single `config/urls.py` include line per app is added in Phase 0 up front, so even the router isn't touched in parallel.

Why the same variables are guaranteed

Both import the identical symbols — e.g. both write `LedgerEvent.STAMP`, `CustomerCard.stamp_count`, `customer_card_id` — because those names exist in exactly **one** place (`core/`), and \$3 froze them. Neither of you invents `stamps`, `stampCount`, or `card_id`.

The one seam, handled by interface

Joe's `loyalty/stamp` needs to push a wallet update, which *you* implement. Joe codes against `wallets.service.push_update` (defined as a stub in Phase 0) and uses a **fake** in tests:

```
# loyalty/views.py (Joe)
from core import ledger
from wallets import service as wallet    # the façade — stub now, real later

def post(self, request):
    card = get_scoped(CustomerCard, request, id=request.data["customer_card_id"])
    ledger.add_stamp(card, staff=request.staff, location=request.staff.location)
    wallet.push_update(card)              # no-op stub until your Phase 1 lands
    return Response({ "customer_card_id": str(card.id),
                      "stamp_count": card.stamp_count,
                      "stamps_required": card.card.stamps_required,
                      "reward_ready": ledger.is_reward_ready(card) })
```

When your Phase 1 merges, `push_update` becomes real — **Joe's code doesn't change a line**. That's the payoff of freezing the interface in Phase 0.

Merge ritual (every phase end)

1. Rebasing your branch on `main`; open a small PR; the other approves; CI green.
2. Merge both tracks to `main`; deploy to staging.
3. Run the phase exit-criteria smoke together before starting the next phase.

If a contract name must change: stop, open a PR to **this file** changing the name in `core/`, both approve, regenerate `openapi.yaml`, then update callers. Never rename in just one app.

6. Testing & Definition of Done

- **Unit:** `core/ledger.py` (balance math, cooldown, reward-ready) and anti-fraud — the highest-risk logic.
- **Contract tests:** every endpoint response validated against `contracts/openapi.yaml`.
- **Interface tests:** `loyalty/` against a fake `WalletUpdater`; `wallets/` against mocked Google/Apple.
- **Smoke (per platform):** enroll → stamp → live update → redeem.
- **DoD (task):** tests pass in CI · matches the contract · ruff+black+mypy clean · shared-interface change documented · deployed to staging & smoke-tested.
- **DoD (phase):** exit criteria pass on staging with both tracks merged.

7. Appendix — canonical-name cheat sheet

Pin this above your desk. If a name isn't here, it's in §3 — use **that**, don't invent.

Models: `Merchant · Location · StaffUser · Card · CustomerCard · StampLedger · Reward · Redemption · EnrollmentToken · WalletRegistration`

Key fields: `customer_phone · customer_name · stamp_count · stamps_required · auth_token · consent_at · enrolled_at · last_event_at · event_type · delta · balance_after · google_class_id · device_library_id · push_token · google_object_id`

Enums: `Role{OWNER,ADMIN,SCANNER} · MerchantStatus{ACTIVE,SUSPENDED} · PlanTier{FREE,STARTER,GROWTH,CHAIN} · CardType{STAMP,POINTS} · CardStatus{DRAFT,ACTIVE,ARCHIVED}`

CustomerCardStatus{ACTIVE,COMPLETED,BLOCKED} · LedgerEvent{ENROLL,STAMP,REDEEM,ADJUST} ·

WalletPlatform{APPLE,GOOGLE} · RedemptionStatus{PENDING,CLAIMED,VOID}

Ledger fns: record_enrollment · add_stamp · redeem_reward · current_balance · is_reward_ready

Wallet façade: wallets.service.provision · wallets.service.push_update

API roots: /api/v1/auth · /enroll · /loyalty · /cards · /customers · /staff · /locations · /analytics ·
/wallet/apple

JSON keys: customer_card_id · stamp_count · stamps_required · reward_ready · apple_pass_url ·
google_save_url · reward_id · redemption_id

Celery tasks: wallets.tasks.provision_pass · wallets.tasks.push_pass_update ·

wallets.tasks.sync_google_class · core.tasks.recompute_balance · messaging.tasks.send_whatsapp

Error codes: VALIDATION_ERROR · UNAUTHENTICATED · PERMISSION_DENIED · NOT_FOUND · CONFLICT ·
ALREADY_ENROLLED · TOKEN_EXPIRED · COOLDOWN_ACTIVE · RATE_LIMITED · REWARD_NOT_READY ·
WALLET_PROVISION_FAILED · SERVER_ERROR

End of document. Phase 0 freezes everything above; Phases 1 (You) and 2 (Joe) run in parallel against it. Rename “Joe” if your collaborator differs.